Computer Vision Report

Emoji provider classification

Lara Normand, Kenza Gueddas, Louis Bettelli, Oscar Moulet

September–October 2026

Kaggle team: GueddasNormandMouletBettelli Best Kaggle score: 0.96861

Contents

1 Introduction **3**

2 Data **3**

3 Preprocessing **3**
3.1 Image Loading and Normalization 3
3.2 Feature Engineering (222 Features) 4

4 Architecture of the Model **4**
4.1 Model Configuration and Hyperparameters 5
4.2 Histogram-Based Splitting Rationale 5

5 Metrics Chosen **6**

6 Performance **6**
6.1 Validation Protocol and Results 6
6.2 Class-Level Diagnostic Analysis 7
6.3 Comparative Baseline Benchmark 8

7 Limitations and Next Steps **8**

8 Code and AI **9**
8.1 Code Architecture and Reproducibility 9
8.2 Statement on AI Assistance 10

1 Introduction

In this exercise, our mission was to identify the visual origin of each emoji (e.g., Apple, Mozilla...).

We worked on the emoji image dataset provided for this course on Kaggle. Each emoji comes from one of seven providers (Apple, Facebook, Google, Messenger, Mozilla, Samsung, WhatsApp); the challenge is to infer which provider produced the emoji from the image alone. Since the same emoji exists for every provider, the model cannot rely on *what* the emoji represents and has to pick up on *how* it was made.

2 Data

The training set contains 9,879 labelled PNG images, and the classes are imbalanced: Mozilla has about five times fewer images than Apple (Table 1). Looking at the raw files, each provider also has a very recognisable export format. WhatsApp images are almost all 160 px wide, Samsung mostly uses palette mode (P) with very small files, and Facebook, Messenger and Mozilla export at 240 px. These file properties turned out to be very informative, which shaped both our features (Section 3.2) and the limitations we discuss in Section 7.

Table 1: Training set by provider: number of images, most common original width, most common colour mode, and median file size.

Provider	Images	Main width	Main mode	Median size (KB)
Apple	1,924	72 px (100%)	RGBA (94%)	7.1
Facebook	1,667	240 px (93%)	RGBA (99%)	36.4
Google	1,877	72 px (100%)	RGBA (98%)	5.3
Messenger	580	240 px (99%)	RGBA (99%)	21.3
Mozilla	397	240 px (91%)	RGBA (95%)	7.6
Samsung	1,790	72 px (100%)	P (88%)	2.5
WhatsApp	1,644	160 px (90%)	RGBA (97%)	15.0
Total	9,879			

3 Preprocessing

The preprocessing pipeline converts raw emoji images into a structured 222-dimensional feature vector while preserving vital disk-level metadata.

3.1 Image Loading and Normalization

To ensure consistent extraction across heterogeneous image formats, incoming files undergo three normalization steps executed via the `load_rgb` function (used for the colour

histograms of Block C):

- **Format Harmonization:** Images are systematically converted to the RGBA color mode, regardless of their native layout (e.g., palette P, grayscale L, or standard RGB).
- **Alpha Composition:** Transparent background pixels are blended over a solid white background using `Image.alpha_composite`. This eliminates arbitrary hidden color artifacts in transparent regions.
- **Spatial Resizing:** Images are resized to a uniform spatial resolution of 96×96 pixels using a high-quality LANCZOS anti-aliasing filter.

3.2 Feature Engineering (222 Features)

Rather than passing raw pixel grids directly into tree structures, the model summarizes each image across three specialized feature blocks:

- **Block A – File Metadata (15 features):** Extracted from the raw file prior to spatial transformation to capture vendor export signatures. This block includes logarithmic transformations of original width, original height, aspect ratio, and file size on disk (log(width), log(height), ratio, log(file size)). It also records opacity statistics (mean alpha, standard deviation of alpha, fraction of visible pixels with $\alpha > 0.05$), bounding box dimensions ($\text{bbox}_{\text{width}}$, $\text{bbox}_{\text{height}}$, $\text{bbox}_{\text{area}}$), and one-hot encoded flags for original image modes (RGBA, RGB, P, LA, L).
- **Block B – Palette Summary Statistics (15 features):** Computed exclusively over visible pixels ($\alpha > 0.05$) of the original image, across the Red, Green, and Blue channels. For each channel, 5 summary statistics are derived: mean, standard deviation, and three quantiles (10%, 50%, and 90%).
- **Block C – Color Histograms (192 features):** Normalized histograms dividing color values into 32 equal intervals across 6 color channels: Red, Green, Blue (RGB) and Hue, Saturation, Value (HSV) (6 channels $\times$ 32 bins = 192 features). Incorporating the HSV space allows the model to decouple chromatic hue from saturation and overall brightness.

4 Architecture of the Model

The core architecture utilizes a `HistGradientBoostingClassifier` from scikit-learn, chosen for its speed and capacity to model non-linear interactions across tabular features.

4.1 Model Configuration and Hyperparameters

The model is an ensemble of decision trees constructed through iterative gradient boosting:

- **Boosting Iterations (`max_iter`):** Set to 350 rounds.
- **Multi-Class Boosting:** Since there are 7 vendor classes, 7 distinct trees are constructed at each round to update class logit scores, resulting in $350 \times 7 = 2{,}450$ total decision trees.
- **Tree Complexity (`max_leaf_nodes`):** Capped at 31 leaves per tree, constraining individual base learners to prevent overfitting.
- **Learning Rate (`learning_rate`):** Configured to 0.08 to shrink the contribution of each newly added tree.
- **Regularization (`l2_regularization`):** Applied with a factor of 0.1 to stabilize leaf weight corrections.
- **Class Weighting (`class_weight`):** Configured to `"balanced"`. This automatically weights each training sample inversely proportional to its class frequency, ensuring minority classes (e.g., Mozilla with 397 samples) receive balanced gradient updates relative to dominant classes (e.g., Apple with 1,924 samples).

The first four values are set in `configs/style_boosting.yaml`; `class_weight` is fixed directly in `model.py`.

How we chose these values. We did not tune these hyperparameters systematically. The values were suggested by the AI assistant we used while writing the code, and we kept them because the model already reached 0.960 on validation. They stay close to the scikit-learn defaults (31 leaves, light regularization), with a slightly lower learning rate (0.08 instead of 0.1) compensated by more rounds (350 instead of 100). A proper search over these values (for example a grid search on the validation set) is part of our next steps.

Note that scikit-learn only enables early stopping automatically above 10,000 training samples, so with our 7,903 (and later 9,879) images all 350 rounds are always run.

4.2 Histogram-Based Splitting Rationale

Before training, continuously valued features in the dataset are discretized into up to 255 integer-valued bins. When determining optimal node splits, decision trees evaluate split candidates across these pre-computed bin boundaries rather than sorting all individual training values. This histogram technique drastically reduces training complexity while maintaining predictive accuracy. These 255 bins are unrelated to the 32 colour bins of Block C: the former only speed up training, the latter are features.

5 Metrics Chosen

To evaluate performance across the 7 imbalanced provider classes, two primary evaluation metrics were selected:

- **Accuracy:** The overall proportion of correctly classified emojis across the entire dataset.
$$\text{Accuracy} = \frac{\sum_{c=1}^{7} TP_c}{N_{\text{total}}}$$
- **Macro-Averaged F1-Score (Macro-F1):** The unweighted arithmetic mean of individual F1-scores calculated independently for each vendor class.
$$\text{Macro-F1} = \frac{1}{7} \sum_{c=1}^{7} F1_c$$
This metric treats every class with equal importance, preventing high performance on large classes (e.g., Apple) from masking poor accuracy on underrepresented classes (e.g., Mozilla).

In addition to scalar metrics, class-level Precision, Recall, and a 7×7 Confusion Matrix were computed to diagnose specific pairwise vendor misclassifications.

6 Performance

6.1 Validation Protocol and Results

The evaluation was performed using a stratified 80/20 train/validation split (7,903 training samples and 1,976 hold-out validation samples) with a fixed seed (`random_state=42`). Final predictions were generated after retraining the model on 100% of the training data (9,879 images).

On the unseen validation set of 1,976 images, the model correctly identified 1,897 emojis and committed 79 errors, achieving an overall Accuracy of 0.960 and a Macro-F1 score of 0.965. On the Kaggle test set, our best submission scored 0.96861 on the public leaderboard, which is computed on about half of the test data; the final ranking uses the other half. This is in line with our validation accuracy.

6.2 Class-Level Diagnostic Analysis

Table 2: Validation confusion matrix (rows: true provider, columns: predicted). Diagonal = correct predictions. FB = Facebook, Msgr = Messenger, WA = WhatsApp.

	Apple	FB	Google	Msgr	Mozilla	Samsung	WA
Apple	**366**	1	17	0	0	0	1
Facebook	12	**317**	3	1	0	0	0
Google	9	0	**367**	0	0	0	0
Messenger	0	4	0	**112**	0	0	0
Mozilla	0	0	0	2	**77**	0	0
Samsung	9	0	2	0	0	**347**	0
WhatsApp	14	0	0	0	0	4	**311**

- All seven providers achieved class-level F1-scores exceeding 0.920.
- Mozilla attained near-perfect accuracy (Precision 1.0; only 2 of its 79 validation images misclassified), possessing distinct export signatures.
- Apple recorded the lowest Precision (0.893). It acts as a primary false-positive sink, absorbing 44 of the total 79 validation errors (including 14 misclassified WhatsApp images, 12 Facebook, 9 Google, and 9 Samsung).
- Apple and Google exhibited symmetric confusion (17 Apple predicted as Google, 9 Google predicted as Apple). This is driven by shared baseline metadata properties, such as matching native 72px image widths and similar compressed file sizes.

Figure 1 shows four validation errors, produced by `scripts/misclassified.py`. All four can be explained by file properties rather than by the drawing itself:

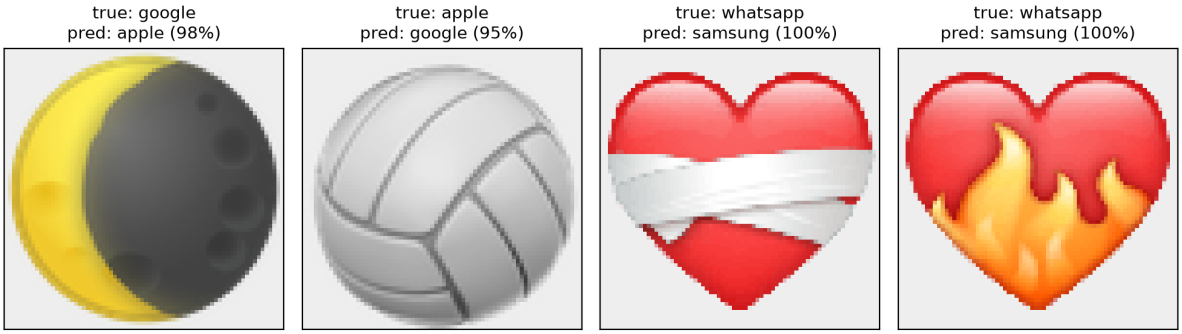


Figure 1: Four misclassified validation images (true and predicted provider, with the model's confidence).

- **Google predicted as Apple (98%):** a 72 px RGBA image like most Apple and Google emojis, but its file size (6.3 KB) is closer to the typical Apple file (median 7.1 KB) than to the typical Google file (5.3 KB).

- **Apple predicted as Google (95%):** saved in grayscale-with-alpha mode (LA), which is about twice as frequent among Google images as among Apple images.
- **Two WhatsApp images predicted as Samsung (100%):** both are 72 px, palette mode (P) and only 2–3 KB, which is exactly Samsung's export signature. The model is fully confident and wrong, which shows how strongly it relies on file format.

6.3 Comparative Baseline Benchmark

The proposed HistGradientBoosting model was benchmarked against the other approaches we tried, on the exact same validation split:

Table 3: Validation accuracy of the approaches we tried (same 80/20 split, seed 42).

Model / feature set	Val. acc.	Kaggle	Comment
LeNet CNN	0.426	–	baseline, trained from scratch
Pretrained MobileNetV3	0.658	–	only sees resized pixels
HOG + colour + log. regression	0.757	–	shape + colour, linear model
Log. regression (colour histograms)	0.780	–	colour only
HistGB without Block A (207 features)	0.866	–	ablation: colour only
Extra Trees (222 features)	0.936	–	metadata + colour
HistGB (222 features, ours)	**0.960**	**0.969**	**final model**
HistGB + MobileNetV3 ensemble	0.963	–	+0.003, not kept

Kaggle = public leaderboard score; "–" = model not submitted. Our team made two submissions: a first exploratory one (0.79044) and our final HistGB model (0.96861).

The CNNs only receive the resized image, so they cannot use the original width, colour mode or file size, which are very discriminative in this dataset (Table 1). MobileNetV3 scoring below a simple colour-histogram model is still surprising, and we suspect our preprocessing for it (input range or transparency handling) was not optimal. The ensemble with MobileNetV3 gained only 0.003 in validation accuracy, so we kept the single HistGB model, which is simpler to explain, to run and to maintain.

7 Limitations and Next Steps

- **Format Leakage vs. Pure Visual Style:** A minimal 3-level tree operating strictly on file width, image mode, and file weight already reaches 76% accuracy on the training set without reading any color values. To measure this properly, we retrained the same model without Block A (207 features, same split): validation accuracy drops from 0.960 to 0.866 (macro-F1 from 0.965 to 0.882), i.e. the number of errors roughly triples. File metadata therefore accounts for a real part of our score, but the colour features alone still reach 0.866, well above our other colour-only and CNN baselines (Table 3). Our model predicts "the provider as it appears in this dataset",

not pure visual style: if a provider changed its export format, the model would likely lose accuracy where a human would not.

- **Dataset Duplicate Overlap:** Hashing analysis identified 132 duplicate image groups (281 images) in the training set and 293 test images identical to training images. Random train/val splitting allowed identical duplicates to fall on both sides, making validation metrics slightly optimistic. The test duplicates likely inflate the Kaggle score in the same way.

Next steps.

- Remove exact duplicates and split by group of identical images, for example with scikit-learn's `GroupShuffleSplit`, to get an honest validation score.
- Add shape and texture features (e.g. HOG) to close the gap between the model without metadata (0.866) and the full model (0.960).

8 Code and AI

8.1 Code Architecture and Reproducibility

The codebase is structured into a modular Python package adhering to software engineering best practices:

- `configs/style_boosting.yaml`: Image size, histogram bins, split settings and the four main boosting hyperparameters.
- `src/emoji_provider/data.py`: Handles image dataset loading and matches each image to its label by Id.
- `src/emoji_provider/features.py`: Implements the feature extraction pipeline (Block A, B, and C vectors).
- `src/emoji_provider/model.py`: Builds the gradient boosting classifier, including `class_weight`.
- `scripts/run.py`: Executes model training, logs validation metrics, and exports timestamped submission files.
- `scripts/ablation.py`: Retrains the model with and without Block A on the same split (Section 7).
- `scripts/misclassified.py`: Saves examples of misclassified validation images (Figure 1).
- `tests/test_features.py`: Unit test suite ensuring exact dimensional alignment of the 222 features via `pytest`.

Environment setup and dependencies are managed via `uv` (`uv sync`; `uv run python scripts/run.py`), with all library versions pinned. The code requires Python 3.11 or 3.12. The data must be downloaded from the Kaggle competition page and placed in `data/train/`, `data/test/` and `data/train_labels.csv`.

8.2 Statement on AI Assistance

We used Claude (Anthropic) for code scaffolding, unit tests, configuration parsing and LaTeX formatting. The hyperparameters were also suggested by Claude (Section 4.1). Claude helped build the explanatory website in `site/`, which walks through the whole project with diagrams, interactive demos and embedded videos (open with `python3 -m http.server -d site 8000`). It was also used to review this report against the code and to write the ablation and error-analysis scripts. Prompts are listed in `PROMPTS.md`; all results were checked by the team.